

# GCP DAY 4: DEEP DIVE TECHNICAL TUTORIAL

## Cloud IAM, Terraform, Cloud Marketplace & Cloud Source Repositories

---

### TABLE OF CONTENTS

1. [Cloud IAM: Architecture & Strategy](#)
  2. [Infrastructure as Code with Terraform](#)
  3. [Provision Services with Google Cloud Marketplace](#)
  4. [Cloud Source Repositories](#)
  5. [Integration Patterns](#)
  6. [Enterprise Implementation](#)
- 

## CLOUD IAM: ARCHITECTURE & STRATEGY

### 1. Conceptual Framework

#### Identity Model

Google Cloud IAM operates on a **who-what-where** principle:

##### WHO (Principal/Identity)

- └─ User accounts
- └─ Service accounts
- └─ Google groups
- └─ Cloud identities

##### WHAT (Role/Permissions)

- └─ Basic roles (deprecated)
- └─ Predefined roles
- └─ Custom roles
- └─ Conditional roles

##### WHERE (Resource)

- └─ Projects
- └─ Folders
- └─ Organization
- └─ Individual resources
- └─ Custom hierarchies

### The IAM Policy Structure

An IAM policy is fundamentally a binding mechanism consisting of:

```
json
```

```
{
  "bindings": [
    {
      "role": "roles/compute.admin",
      "members": [
        "serviceAccount:my-app@project.iam.gserviceaccount.com"
      ],
      "condition": {
        "expression": "resource.matchTag('environment', 'prod')",
        "title": "Production only",
        "description": "This binding applies only to production resources"
      }
    }
  ]
}
```

## 2. Identity Types: Deep Analysis

### User Accounts

#### Characteristics:

- Represent real humans
- Associated with a Google account (Gmail or Cloud Identity)
- Can have multiple projects/organizations
- Inherit permissions from groups and direct assignment

#### Use Cases:

- Development team members
- DevOps engineers
- Application owners
- Auditors

#### Lifecycle Considerations:

- Created when user first logs in or is invited
- Can be disabled without deletion
- Retention policies vary by organization
- Offboarding requires permission cleanup

#### Best Practice Pattern:

---

Avoid: Direct assignment to individuals

Prefer: Group-based assignment

Individual User

↓ (member of)

Google Group

↓ (granted role to)

Project/Folder

↓ (inherits permissions)

Secured Resource

## Service Accounts

### Architecture:

- Machine identities designed for applications
- Have unique email: `name@PROJECT_ID.iam.gserviceaccount.com`
- Generate cryptographic keys (Google-managed or user-managed)
- Support impersonation by other service accounts

### Types:

#### 1. Default Service Account

- Automatically created per project
- Has broad permissions by default (not recommended for production)
- Often used inadvertently
- Best practice: Disable or remove default permissions

#### 2. User-Created Service Accounts

- Specific purpose per account
- Recommended for all workloads
- Enable granular permission control
- Can be scoped to specific resources

### Service Account Lifecycle:

---

Create SA

↓

Generate Key (optional)

↓

Grant Roles via IAM Policy

↓

Use in Application/Service

↓

Rotate Keys (every 90 days recommended)

↓

Disable Old Keys

↓

Delete When Decommissioned

## Key Management Patterns:

Approach 1: Google-Managed Keys (Recommended for modern apps)

|                                        |  |
|----------------------------------------|--|
| Application                            |  |
| ─ Workload Identity (Kubernetes)       |  |
| ─ Workload Identity Federation (CI/CD) |  |
| ─ Impersonation from another service   |  |

Approach 2: User-Managed Keys (Legacy, short-term use)

|                                   |  |
|-----------------------------------|--|
| Application                       |  |
| ─ JSON key file (stored securely) |  |
| ─ P12 key file (older standard)   |  |
| ─ Manual key rotation required    |  |

## 3. Role Architecture

### Role Hierarchy

```
| CUSTOM ROLES (Most Specific)
| └─ tailored-api-access
|   └─ custom-monitoring-viewer
```

↑

```
| PREDEFINED ROLES (Recommended for Production)
| └─ roles/compute.admin          (Compute resources)
| └─ roles/storage.objectViewer  (Cloud Storage)
| └─ roles/run.deployer          (Cloud Run)
| └─ roles/cloudfunctions.admin  (Cloud Functions)
|   └─ roles/iam.securityAdmin   (IAM management)
```

↑

```
| BASIC ROLES (Legacy - Avoid in Production)
| └─ roles/owner
| └─ roles/editor
|   └─ roles/viewer
```

## Understanding Permissions

A permission is an atomic authorization that grants ability to perform a specific action:

### Permission Naming Convention:

```
service.resource.verb
```

Examples:

```
└─ compute.instances.get      (Read single instance)
└─ compute.instances.list    (List instances)
└─ compute.instances.create  (Create instance)
└─ compute.instances.delete  (Delete instance)
└─ storage.buckets.create    (Create bucket)
└─ storage.objects.get       (Read object)
└─ iam.roles.get             (Read role definition)
  └─ logging.logEntries.create (Write to logs)
```

### Permission Grouping in Roles:

```
Role: roles/compute.admin
```

- └─ compute.\* (all compute permissions)
- └─ compute.instances.\* (all instance operations)
- └─ compute.disks.\* (all disk operations)
- └─ compute.networks.\* (all network operations)

```
Role: roles/storage.objectViewer
```

- └─ storage.buckets.get
- └─ storage.buckets.list
- └─ storage.objects.get
- └─ storage.objects.list

## 4. Custom Roles

### When to Create Custom Roles

#### Scenario 1: Least Privilege by Service

Goal: Give application only permissions it needs

Traditional: Grant roles/compute.admin

Custom: Create role with only:

- └─ compute.instances.get
- └─ compute.instances.list
- └─ compute.disks.get

#### Scenario 2: Cross-Service Permissions

Goal: Monitoring service needs access to multiple services

Application monitors:

- └─ Compute instances
- └─ Cloud SQL databases
- └─ Cloud Storage buckets
- └─ Cloud Run services

Custom role: monitoring-reader

Contains permissions from multiple services

#### Scenario 3: Conditional Permissions

Goal: Specific permissions only for production resources

Custom role with conditions:

```
resource.matchTag('environment', 'prod')  
resource.matchTag('department', 'finance')
```

## Custom Role Structure

yaml

```
title: "API Gateway Manager"
description: "Manage API Gateway with limited scope"
stage: "ALPHA" # ALPHA, BETA, GA, DISABLED

includedPermissions:
  # API Gateway specific
  - apigateway.apiconfigs.create
  - apigateway.apiconfigs.delete
  - apigateway.apiconfigs.get
  - apigateway.apiconfigs.list
  - apigateway.apis.create
  - apigateway.apis.delete
  - apigateway.apis.get
  - apigateway.apis.list

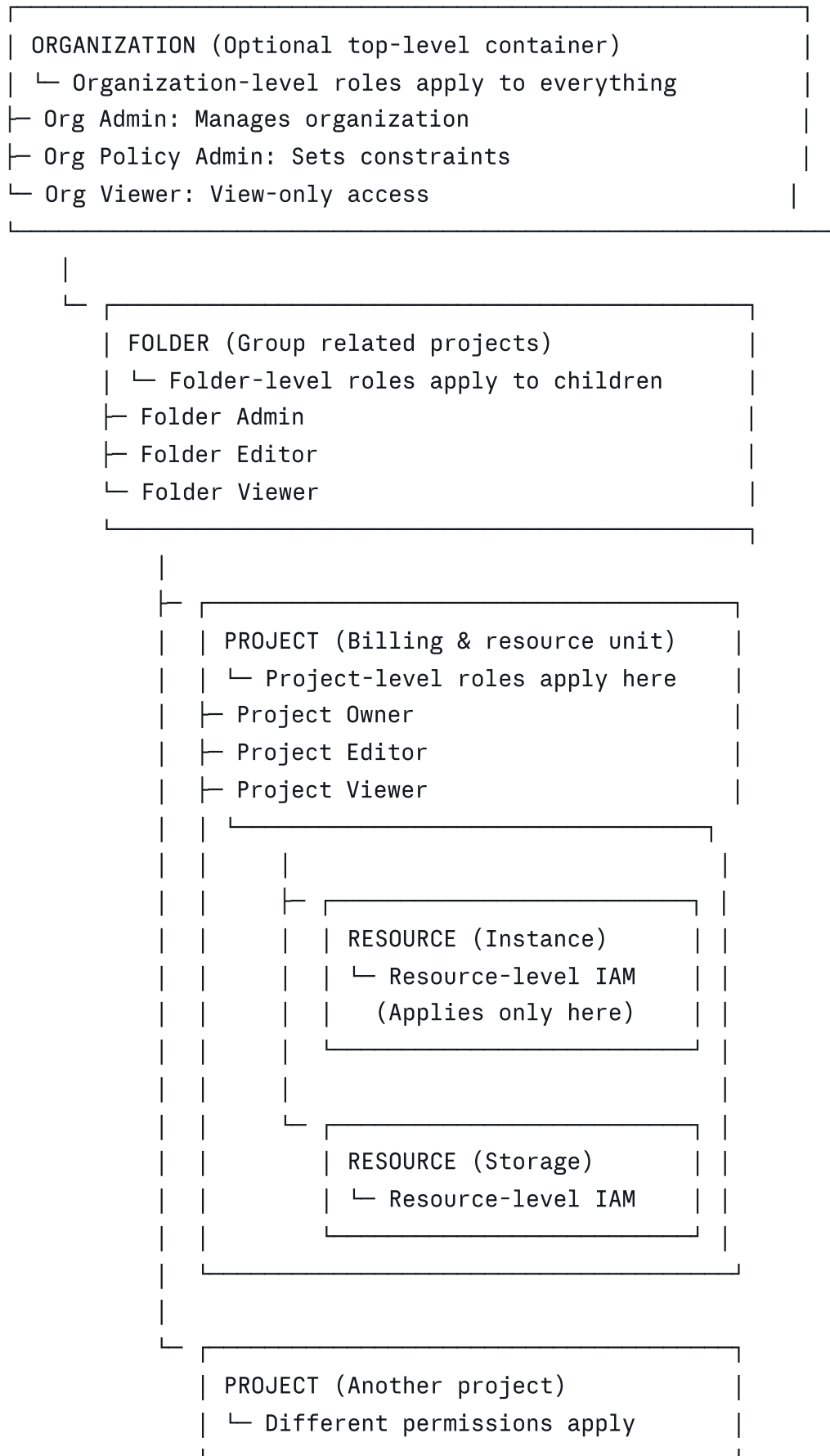
  # Supporting permissions
  - iam.serviceAccounts.actAs # To deploy with SA
  - iam.serviceAccounts.getAccessToken

  # Monitoring
  - monitoring.timeSeries.create
  - logging.logEntries.create
```

## 5. Resource Hierarchy & Inheritance

### Hierarchy Structure

---



## Permission Inheritance Rules

Resource-Level Policy

↑

(More specific)

|

Project Policy

↑

(Can be overridden)

|

Folder Policy

↑

(Can be overridden)

|

Organization Policy

↑

(Least specific)

PRINCIPLE: More specific policy overrides less specific

## Inheritance Example:

Organization: org-admin@company.com → roles/viewer

└─ Can see all resources

Folder Finance: finance-team@company.com → roles/editor

└─ Can edit finance resources only

Project: payment-processing

No additional bindings

Results in:

└─ org-admin@company.com → roles/viewer (from org)

└─ finance-team@company.com → roles/editor (from folder)

## 6. Conditional IAM

### Access Context Conditions

Conditions restrict role access based on context:

yaml

```
role: roles/compute.admin
members:
  - serviceAccount: deployer@project.iam.gserviceaccount.com
condition:
  expression: |
    resource.matchTag('environment', 'prod') &&
    resource.matchTag('department', 'platform')
  title: "Production Platform Only"
  description: "Only grant access to production platform resources"
```

## Available Context Attributes

Resource attributes:

- └ resource.name
- └ resource.type
- └ resource.matchTag('key', 'value')
- └ resource.matchTagPrefix('key', 'prefix')

Request attributes:

- └ request.time
- └ request.auth.principal
- └ request.auth.claims
- └ request.auth.audiences

Environment attributes:

- └ google.compute.zone
- └ google.compute.region

## Real-World Example:

yaml

```
# Scenario: CI/CD can only deploy to staging on weekdays 9-5

role: roles/run.deployer
members:
  - serviceAccount: ci-cd@project.iam.gserviceaccount.com

condition:
  expression: |
    (
      request.time.getHours('America/New_York') >= 9 &&
      request.time.getHours('America/New_York') <= 17 &&
      request.time.getDayOfWeek('America/New_York') >= 1 &&
      request.time.getDayOfWeek('America/New_York') <= 5
    ) ||
    resource.matchTag('environment', 'staging')
```

## 7. IAM Best Practice Patterns

### Pattern 1: Least Privilege by Function

Application Architecture:

- └ API Service (handles requests)
- └ Database Migration (updates schema)
- └ Backup Service (reads data)
- └ Monitoring Agent (reads metrics)

Service Account Structure:

- └ api-service-sa
  - └ Roles: compute.admin, storage.objectViewer
  - └ Permissions: Only for compute/storage operations
- └ db-migration-sa
  - └ Roles: cloudsql.admin (custom limited)
  - └ Permissions: Only schema updates
- └ backup-service-sa
  - └ Roles: storage.objectCreator (for backups)
  - └ Permissions: Read SQL, write to backup buckets
- └ monitoring-agent-sa
  - └ Roles: monitoring.metricWriter
  - └ Permissions: Write metrics only

### Pattern 2: Environment-Based Access

#### Directory Structure:

```
└─ development/
  │   └─ dev-team@company.com → roles/editor
  │       (Full access in dev)
  │
  └─ staging/
    │   └─ dev-team@company.com → roles/viewer
    │       (Read-only in staging)
    │
    └─ production/
      │   └─ platform-team@company.com → roles/viewer
      │       (All platform team can view prod)
      │
      └─ platform-lead@company.com → roles/editor
          (Only leads can modify prod)
```

### Pattern 3: Cross-Organization Access

#### Organization A (Publisher)

```
└─ Service: analytics-service-sa
└─ Publishes to: shared bucket
```

#### Organization B (Consumer)

```
└─ Service: consumer-app-sa
└─ Needs access to shared bucket
  └─ Cannot use local service account
```

#### Solution:

1. Org A creates shared bucket
2. Org A grants access to Org B's service account
3. Use fully qualified principal:  
serviceAccount:consumer-app@orgB.iam.gserviceaccount.com

### Pattern 4: Approval/Workflow-Based Access

Scenario: Production deployments require approval

Step 1: Developer in dev-deployer group

- └ Can deploy to staging only

Step 2: Request production deployment

- └ Creates ticket in approval system

Step 3: Approver reviews and approves

- └ Temporarily adds deployer to prod-deployer group

Step 4: Deployment happens

- └ Deployer uses prod permissions

Step 5: Automatic removal after 24 hours

- └ Deployer removed from prod-deployer group

---

## INFRASTRUCTURE AS CODE WITH TERRAFORM

### 1. Terraform Conceptual Model

#### State Management Philosophy

Terraform maintains a state file representing actual infrastructure:

Desired State (\*.tf files)

↓

- └ terraform plan

- | └ Compares with

- |

- └> Current State (terraform.tfstate)

- | └ Track of what's deployed

- |

- └> Remote State (Cloud Storage)

- └ Shared state for teams

#### State File Structure:

json

```

{
  "version": 4,
  "terraform_version": "1.5.0",
  "serial": 42,
  "lineage": "unique-id",
  "outputs": {},
  "resources": [
    {
      "type": "google_compute_instance",
      "name": "example",
      "instances": [
        {
          "attributes": {
            "id": "instance-1234",
            "name": "my-instance",
            "machine_type": "e2-medium"
          }
        }
      ]
    }
  ]
}

```

## State Locking for Concurrent Operations:

Team Member 1: terraform apply

- ├─ Acquires state lock (timeout: 30s)
- ├─ Modifies infrastructure
- └─ Releases lock

Team Member 2: terraform apply (concurrent)

- └─ Waits for lock → "Another terraform process is running"
- After member 1 releases → Can proceed

## 2. Provider Architecture

### Provider Configuration Layers

```
| Terraform Configuration (Explicit) |  
| └─ provider "google" { credentials = "..."} |  
| └─ Sets explicit authentication |
```

↓ (If not explicit, try)

```
| Environment Variables (Semi-automatic) |  
| └─ GOOGLE_APPLICATION_CREDENTIALS |  
| └─ GOOGLE_PROJECT |  
| └─ GOOGLE_REGION |
```

↓ (If not set, try)

```
| gcloud Credentials (Automatic) |  
| └─ Uses `gcloud auth application-default login` |  
| └─ Default authentication for SDK |
```

↓ (If all fail)

ERROR: No valid credentials found

## Multi-Provider Configuration

hcl

```
# Configure multiple regions
provider "google" {
  alias    = "us_east"
  project  = "my-project"
  region   = "us-east1"
}

provider "google" {
  alias    = "us_west"
  project  = "my-project"
  region   = "us-west1"
}

provider "google" {
  alias    = "europe"
  project  = "my-project"
  region   = "europe-west1"
}

# Use specific provider for resources
resource "google_compute_network" "us_east_network" {
  provider = google.us_east
  name     = "us-east-network"
}

resource "google_compute_network" "us_west_network" {
  provider = google.us_west
  name     = "us-west-network"
}
```

### 3. Resource Modeling

#### Resource Declaration Patterns

##### Pattern 1: Simple Resource

hcl

```
resource "google_compute_instance" "web" {
  name          = "web-server"
  machine_type  = "e2-medium"
  zone          = "us-central1-a"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-11"
      size  = 20
    }
  }

  network_interface {
    network = google_compute_network.default.name
    access_config {} # Public IP
  }

  metadata = {
    enable-oslogin = "TRUE"
  }

  tags = ["web", "http-server"]
}
```

## Pattern 2: Resource with Dependencies

hcl

```
resource "google_storage_bucket" "app_bucket" {
  name          = "${var.project_id}-app-data"
  location      = var.region
  force_destroy = false

  # Explicit dependency
  depends_on = [
    google_project_iam_member.bucket_access
  ]
}

resource "google_project_iam_member" "bucket_access" {
  project = var.project_id
  role    = "roles/storage.admin"
  member  = "serviceAccount:${google_service_account.app.email}"

  # Implicit dependency on service account
}

resource "google_service_account" "app" {
  account_id   = "app-sa"
  display_name = "Application Service Account"
}
```

### Pattern 3: Computed Values & Dynamic Blocks

hcl

```

resource "google_compute_instance" "dynamic_instance" {
  name          = "instance-${count.index}"
  machine_type  = var.machine_type
  zone          = var.zone

  # Dynamic network interfaces based on variable
  dynamic "network_interface" {
    for_each = var.networks
    content {
      network          = network_interface.value.name
      access_config = network_interface.value.public_ip ? [{}] : []
    }
  }

  # Metadata from local values
  metadata = local.common_tags

  # Conditionally add service account
  service_account {
    email  = var.enable_sa ? google_service_account.app[0].email : null
    scopes = ["cloud-platform"]
  }
}

```

## 4. Module Architecture

### Module Organization Patterns

#### Pattern 1: Layered Modules

```
modules/
├─ network/                # Network infrastructure
│  ├─ main.tf
│  ├─ variables.tf
│  ├─ outputs.tf
│  └─ vpc.tf
│
├─ compute/                # Compute resources
│  ├─ main.tf
│  ├─ variables.tf
│  ├─ outputs.tf
│  └─ instances.tf
│
├─ storage/                # Storage resources
│  ├─ main.tf
│  ├─ variables.tf
│  ├─ outputs.tf
│  └─ buckets.tf
│
└─ security/               # Security & IAM
   ├─ main.tf
   ├─ variables.tf
   ├─ outputs.tf
   └─ iam.tf
```

## Pattern 2: Application Modules

---

```
modules/
├─ web_app/
│   ├─ main.tf
│   ├─ variables.tf
│   ├─ outputs.tf
│   └─ services/
│       ├─ api_server.tf
│       ├─ database.tf
│       └─ cache.tf
└─ backend_app/
    ├─ main.tf
    ├─ variables.tf
    ├─ outputs.tf
    └─ services/
        ├─ worker.tf
        ├─ queue.tf
        └─ storage.tf
```

### Pattern 3: Multi-Tenant Modules

```
modules/
└─ tenant/
    ├─ main.tf
    ├─ variables.tf
    └─ outputs.tf

# Usage: Each tenant gets isolated resources
module "tenant_acme" {
    source = "../modules/tenant"

    tenant_name = "acme"
    environment = "production"
    tenant_id   = "tenant-001"
}

module "tenant_widgets" {
    source = "../modules/tenant"

    tenant_name = "widgets"
    environment = "production"
    tenant_id   = "tenant-002"
}
```

## Module Composition

```
hcl

# Root module orchestrates everything
module "network" {
  source = "./modules/network"

  vpc_name      = "main-vpc"
  subnet_count  = 3
  region        = var.region
  enable_nat     = true
  enable_vpn     = var.environment == "production"
}

module "compute" {
  source = "./modules/compute"

  instance_count = var.environment == "production" ? 5 : 1
  network        = module.network.network_name
  subnet         = module.network.subnet_names[0]

  depends_on = [module.network]
}

module "security" {
  source = "./modules/security"

  network_id = module.network.network_id
  instances  = module.compute.instance_ids

  enable_oslogin = var.environment == "production"
  audit_enabled  = true
}
```

## 5. State Management Strategies

### Remote State Architecture

```
| Cloud Storage Backend |
| └─ Bucket: project-terraform-state |
| └─ Versioning: Enabled (track all versions) |
| └─ Encryption: Customer-managed keys (CMEK) |
| └─ Access: Limited to service account |
| └─ Locking: DynamoDB (managed by Terraform) |
| └─ State Files: |
|   └─ prod/terraform.tfstate |
|   └─ staging/terraform.tfstate |
|   └─ dev/terraform.tfstate |
```

## Backend Configuration:

```
hcl

terraform {
  backend "gcs" {
    bucket = "project-terraform-state"
    prefix = "prod"

    # Additional security
    encryption_key = "base64-encoded-key"
  }
}

# Environment-specific backends
# backend-dev.tfvars → dev/
# backend-staging.tfvars → staging/
# backend-prod.tfvars → prod/
```

## State Isolation Strategies

### Strategy 1: Workspace-Based Separation

```
bash
```

```
# Create workspaces for environments
terraform workspace new dev
terraform workspace new staging
terraform workspace new prod

# Organize state
dev/terraform.tfstate
staging/terraform.tfstate
prod/terraform.tfstate

# Switch between workspaces
terraform workspace select prod
terraform apply # Applies to prod only
```

## Strategy 2: Directory-Based Separation

```
infrastructure/
├─ dev/
│   ├── main.tf
│   ├── terraform.tfvars
│   └─ backend.tf
│
├─ staging/
│   ├── main.tf
│   ├── terraform.tfvars
│   └─ backend.tf
│
└─ prod/
    ├── main.tf
    ├── terraform.tfvars
    └─ backend.tf

# Each directory is independent state
cd dev && terraform apply
cd staging && terraform apply
cd prod && terraform apply
```

## Strategy 3: Module-Based Separation (Monorepo)

```
# Single state but multiple modules
main.tf:
├─ module "dev" (dev environment)
├─ module "staging" (staging environment)
└─ module "prod" (production environment)

# Risks:
- Single state file couples all environments
- One mistake affects all environments
- Large state file performance issues
```

## 6. Advanced Terraform Patterns

### Pattern 1: Dynamic Resource Creation

hcl

```

# Create multiple instances from list
variable "instances" {
  type = map(object({
    machine_type = string
    zone         = string
    image        = string
  }))
}

resource "google_compute_instance" "instances" {
  for_each = var.instances

  name          = each.key
  machine_type  = each.value.machine_type
  zone          = each.value.zone

  boot_disk {
    initialize_params {
      image = each.value.image
    }
  }
}

# terraform.tfvars:
instances = {
  "web-server-1" = {
    machine_type = "e2-medium"
    zone         = "us-central1-a"
    image        = "debian-11"
  }
  "web-server-2" = {
    machine_type = "e2-medium"
    zone         = "us-central1-b"
    image        = "debian-11"
  }
}

```

## Pattern 2: Conditional Resource Creation

hcl

```

# Deploy resources based on environment
variable "environment" {
  type = string
  validation {
    condition      = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

# Only create in production
resource "google_sql_database_instance" "production_db" {
  count = var.environment == "prod" ? 1 : 0

  name          = "production-db"
  database_version = "POSTGRES_14"

  settings {
    tier              = "db-custom-4-16384"
    availability_type = "REGIONAL"
    backup_configuration {
      enabled              = true
      point_in_time_recovery_enabled = true
      backup_retention_days = 30
    }
  }
}

# Use lower-tier in dev/staging
resource "google_sql_database_instance" "dev_db" {
  count = var.environment != "prod" ? 1 : 0

  name          = "dev-db"
  database_version = "POSTGRES_14"

  settings {
    tier = "db-f1-micro"
    backup_configuration {
      enabled              = false
      backup_retention_days = 7
    }
  }
}

```

### Pattern 3: Data Source Integration

hcl

```
# Reference existing resources without managing them
data "google_compute_network" "default" {
  name = "default"
}

# Use in resource
resource "google_compute_firewall" "allow_http" {
  network = data.google_compute_network.default.self_link

  allow {
    protocol = "tcp"
    ports    = ["80", "443"]
  }
}

# Data source for dynamic values
data "google_client_config" "current" {}

# Get current project
resource "google_service_account" "app" {
  account_id = "app-sa"
  project    = data.google_client_config.current.project
}
```

## Pattern 4: Local Values & Computed Configuration

hcl

```
locals {
  # Environment-specific settings
  env_config = {
    dev = {
      instance_count    = 1
      machine_type       = "e2-small"
      enable_backup      = false
      enable_monitoring = false
      replicas           = 1
    }
    staging = {
      instance_count    = 2
      machine_type       = "e2-medium"
      enable_backup      = true
      enable_monitoring = true
      replicas           = 2
    }
    prod = {
      instance_count    = 5
      machine_type       = "n2-standard-4"
      enable_backup      = true
      enable_monitoring = true
      replicas           = 3
    }
  }

  # Merge common tags
  common_tags = {
    Terraform    = "true"
    Environment   = var.environment
    ManagedBy     = "platform-team"
    CreatedAt     = timestamp()
  }

  # Computed values
  resource_prefix = "${var.project_id}-${var.environment}"

  # Conditional configuration
  backup_settings = local.env_config[var.environment].enable_backup ? {
    enabled           = true
    retention_days     = 30
    point_in_time_recovery = true
  } : {
    enabled = false
  }
```

```
    }  
  }  
  
# Usage  
resource "google_compute_instance" "app" {  
  count          = local.env_config[var.environment].instance_count  
  name           = "${local.resource_prefix}-instance-${count.index + 1}"  
  machine_type   = local.env_config[var.environment].machine_type  
  
  tags = merge(local.common_tags, {  
    Name = "app-instance"  
  })  
}
```

## 7. State Drift & Reconciliation

### Detecting State Drift

```
bash  
  
# Refresh state to detect drift  
terraform refresh  
# Output changes if infrastructure changed outside Terraform  
  
# Plan to see what's different  
terraform plan  
# Shows resources that will be added/changed/deleted
```

### State Drift Scenarios:

### Scenario 1: Manual Changes

User manually resizes instance in console

↓

Terraform plan shows instance size has changed

↓

Options:

- a) Reapply Terraform to restore configured size
- b) Import actual state: terraform import
- c) Update configuration to match actual

### Scenario 2: External Changes

Monitoring service auto-scales instance

↓

Terraform state differs from reality

↓

Solutions:

- a) Use lifecycle.ignore\_changes for specific attributes
- b) Use Compute Engine autoscaling instead
- c) Refresh and reconcile state

### Scenario 3: Failed Deployment

Deployment partially succeeds

↓

State shows success, reality shows failure

↓

Solutions:

- a) terraform taint forces recreation
- b) terraform destroy + reapply
- c) Manual fix + terraform refresh

## Preventing Drift with Lifecycle Rules

hcl

```
resource "google_compute_instance" "app" {
  name          = "app-instance"
  machine_type  = "e2-medium"

  # Don't manage these if changed outside Terraform
  lifecycle {
    ignore_changes = [
      # Allow console changes without triggering updates
      labels,
      metadata,

      # Ignore specific attributes
      tags,
    ]
  }
}

# Prevent accidental deletion
resource "google_sql_database_instance" "critical_db" {
  name = "critical-db"

  lifecycle {
    prevent_destroy = true
  }
}

# Use create_before_destroy for zero-downtime updates
resource "google_compute_instance_template" "app" {
  name = "app-template"

  lifecycle {
    create_before_destroy = true
  }
}
```

---

## PROVISION SERVICES WITH GOOGLE CLOUD MARKETPLACE

### 1. Marketplace Architecture

#### Solution Classification System

---

## GOOGLE CLOUD MARKETPLACE

### CATEGORY 1: VM SOLUTIONS

- └ Preconfigured images
- └ Single-instance deployments
- └ Examples: WordPress, Drupal, Jenkins

### CATEGORY 2: KUBERNETES SOLUTIONS

- └ Container-based applications
- └ Multi-component deployments
- └ Examples: MongoDB, PostgreSQL, Nginx

### CATEGORY 3: MANAGED SERVICES (SaaS)

- └ API integrations
- └ Service subscriptions
- └ Examples: Datadog, PagerDuty, Twilio

### CATEGORY 4: PUBLIC DATASETS

- └ BigQuery datasets
- └ Pre-loaded data
- └ Examples: COVID-19, Climate, Stock prices

### CATEGORY 5: APIs & SERVICES

- └ Third-party APIs
- └ AI/ML services
- └ Examples: AI models, Translation APIs

## 2. VM Solutions Deep Dive

### Deployment Mechanism

User initiates deployment

↓

Marketplace validates

- └ Check project quota
- └ Verify permissions
- └ Confirm billing method

↓

Deployment Manager processes

- └ Creates resource template
- └ Submits to GCP APIs
- └ Monitors deployment

↓

Resources created

- └ Compute Instance
- └ Network interfaces
- └ Firewall rules
- └ Static IP (optional)
- └ Storage (optional)

↓

Deployment complete

- └ Access via IP/DNS

## Configuration Customization Options

### Pre-Deployment Configuration:

Machine Type Selection

- └ Small: e2-small (basic applications)
- └ Medium: e2-medium (standard workloads)
- └ Large: e2-standard-4 (high-performance)
- └ Custom: Define vCPU/memory

Storage Configuration

- └ Boot disk size (10-10000 GB)
- └ Disk type (Standard, Balanced, SSD)
- └ Disk encryption (Google-managed or CMEK)

Network Configuration

- └ VPC network selection
- └ Subnet selection
- └ Public IP (Yes/No)
- └ Network tags
- └ Firewall rules

## Post-Deployment Configuration:

### Application Customization

- └ Default credentials (admin/password)
- └ Database initialization
- └ Plugin installation
- └ Theme/design configuration
- └ DNS/domain setup

### Security Hardening

- └ SSL certificate installation
- └ Firewall rule refinement
- └ OS updates & patches
- └ User access control
- └ Backup configuration

## 3. Kubernetes Solutions

### Deployment Process

```
marketplace → solution selected
    ↓
validation checks
    ├── GKE cluster exists/create
    ├── Required APIs enabled
    ├── Sufficient quota
    └── RBAC permissions
    ↓
kubernetes manifests generated
    ├── StatefulSet/Deployment
    ├── Service definitions
    ├── ConfigMap/Secret
    ├── PersistentVolume
    └── Ingress (if needed)
    ↓
apply to cluster
    ├── kubectl apply
    ├── Wait for pod startup
    └── Configure networking
    ↓
service accessible
    └── Via DNS/Load Balancer
```

### Multi-Component Example (Database)

yaml

```
# MongoDB on Kubernetes (typical marketplace solution)
```

```
# StatefulSet for database
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: StatefulSet
```

```
metadata:
```

```
  name: mongodb
```

```
spec:
```

```
  serviceName: mongodb
```

```
  replicas: 3 # High availability
```

```
  selector:
```

```
    matchLabels:
```

```
      app: mongodb
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: mongodb
```

```
    spec:
```

```
      containers:
```

```
        - name: mongodb
```

```
          image: mongo:5.0
```

```
          ports:
```

```
            - containerPort: 27017
```

```
          volumeMounts:
```

```
            - name: data
```

```
              mountPath: /data/db
```

```
          env:
```

```
            - name: MONGO_INITDB_ROOT_USERNAME
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: mongodb-secrets
```

```
                  key: username
```

```
            - name: MONGO_INITDB_ROOT_PASSWORD
```

```
              valueFrom:
```

```
                secretKeyRef:
```

```
                  name: mongodb-secrets
```

```
                  key: password
```

```
# Persistent storage
```

```
volumeClaimTemplates:
```

```
- metadata:
```

```
  name: data
```

```
spec:
```

```
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 100Gi

# Service for access
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  clusterIP: None
  selector:
    app: mongodb
  ports:
    - port: 27017
      targetPort: 27017
```

## 4. Licensing & Pricing Models

### Licensing Models

---

## LICENSING TYPES

### 1. FREE

- └ No additional costs (e.g., open source images)

### 2. BYOL (Bring Your Own License)

- └ License purchased separately
- └ Provider licenses outside marketplace
- └ Lower infrastructure costs

### 3. USAGE-BASED

- └ Pay for actual usage (storage, API calls)
- └ Monthly or hourly billing
- └ Cost scales with usage

### 4. SUBSCRIPTION

- └ Fixed monthly/annual fee
- └ Unlimited usage within tier
- └ Predictable costs

### 5. HYBRID

- └ Base subscription + usage overage
- └ Combines predictability with scaling
- └ Most expensive per unit, most flexible

## Cost Components

Total Cost = Infrastructure + License + Support

Infrastructure:

- └─ Compute (vCPU × hours × rate)
- └─ Storage (GB × hours × rate)
- └─ Network (GB egress × rate)
- └─ IP addresses (\$ per address/month)

License:

- └─ Included in image
- └─ Charged separately
- └─ Depends on tier/workload

Support:

- └─ Community (free)
- └─ Standard (\$X/month)
- └─ Premium (\$Y/month)

Example: WordPress

Infrastructure: \$20-100/month (depends on size)

License: \$0 (open source)

Support: \$0 (community) or \$50-200/month (premium)

Total: \$20-300/month

## 5. Solution Selection Framework

### Decision Matrix

Requirement: Run Database

Option 1: Self-managed Cloud SQL

- └ Pros: Managed service, automatic backups, scaling
- └ Cons: Vendor lock-in, limited customization
- └ Cost: \$20-500/month
- └ Best for: Standard deployments

Option 2: Marketplace VM (MySQL)

- └ Pros: Full control, customizable
- └ Cons: Manual management, patching
- └ Cost: \$30-200/month
- └ Best for: Custom configurations

Option 3: Marketplace Kubernetes (MongoDB)

- └ Pros: Scalable, high-availability setup
- └ Cons: Complexity, cluster required
- └ Cost: \$50-300/month (+ cluster costs)
- └ Best for: Large-scale deployments

DECISION: Match requirement to option

Small app → Cloud SQL (easiest)

Medium customization → VM

Enterprise scale → Kubernetes

## Evaluation Criteria

- └ Performance Requirements
  - | └ Throughput (QPS, MB/s)
  - | └ Latency (response time)
  - | └ Scalability (growth potential)
- |
- └ Compliance Requirements
  - | └ Data residency
  - | └ Encryption standards
  - | └ Audit logging
- |
- └ Operational Requirements
  - | └ Backup/disaster recovery
  - | └ Monitoring & alerting
  - | └ Support SLA
  - | └ Documentation quality
- |
- └ Integration Requirements
  - | └ API compatibility
  - | └ Data export capability
  - | └ Third-party integrations
- |
- └ Cost Requirements
  - | └ Capital expenditure budget
  - | └ Operating expense budget
  - | └ Total cost of ownership

## 6. Post-Deployment Operations

### Lifecycle Management

---

### Post-Deployment (Day 1-7)

- └ Verify functionality
- └ Change default credentials
- └ Configure custom domain
- └ Enable logging
- └ Initial backup

### Day 7-30

- └ Configure monitoring
- └ Set up automated backups
- └ Create runbooks
- └ Train team
- └ Security audit

### 30+ Days

- └ Monthly patching
- └ Performance tuning
- └ Capacity planning
- └ Cost optimization
- └ Update runbooks

## Operations Patterns

### Pattern 1: Update Management

- Old version → Backup created
  - New version deployed
  - Health check
  - Rollback if failed

### Pattern 2: Backup Strategy

- Daily automated backup
  - └ Retained 7 days locally
  - └ Older backups to cold storage
  - └ RPO: 24 hours, RT0: 4 hours

### Pattern 3: Monitoring

- └ System metrics (CPU, memory, disk)
  - └ Application metrics (response time, errors)
  - └ Business metrics (users, transactions)
  - └ Alerts for thresholds
-

# CLOUD SOURCE REPOSITORIES

## 1. Repository Architecture

### Repository Structure Model

```
Repository
├─ Branches
│   ├── main (production code)
│   ├── develop (integration branch)
│   ├── feature/* (feature development)
│   ├── bugfix/* (bug fixes)
│   └─ release/* (release preparation)
│
├─ Commits (Git objects)
│   ├── SHA hash
│   ├── Author + timestamp
│   ├── Commit message
│   └─ Tree (file structure)
│
├─ Tags (References)
│   ├── v1.0.0 (release tags)
│   ├── stable (branch pointer)
│   └─ production-ready
│
├─ Hooks (Automation)
│   ├── Pre-commit (local)
│   ├── Post-receive (server)
│   └─ Webhooks (Cloud Build)
│
└─ Configuration
    ├── .gitignore (ignored files)
    ├── .gitattributes (file handling)
    └─ .git/config (repository settings)
```

## 2. Git Workflow Models

### Workflow 1: Feature Branch Model (Most Common)

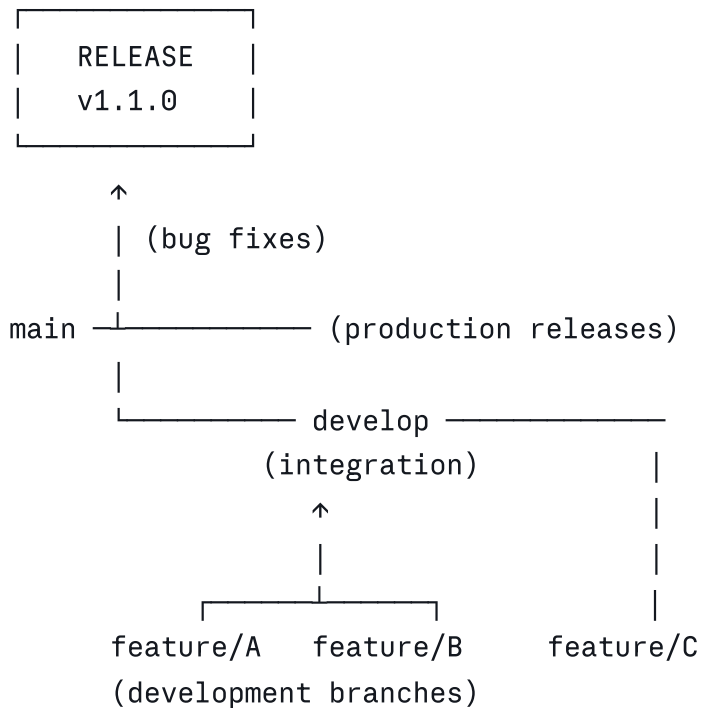
```

main (production)
├─ v1.0.0 (stable release)
│
├─ feature/user-auth
│   ├─ Implement login
│   ├─ Add password hashing
│   └─ Create pull request → review → merge
│
├─ feature/api-versioning
│   ├─ Support v2 API
│   ├─ Deprecate v1
│   └─ Create pull request → review → merge
│
└─ hotfix/security-patch
    ├─ Fix vulnerability
    └─ Create pull request → urgent review → merge

develop (integration)
├─ Merges from feature branches
├─ Released to staging for testing
└─ Merged to main when ready

```

## Workflow 2: Gitflow Model (Enterprise)



## Workflow 3: Trunk-Based Development (Continuous Delivery)

```
main —————  
└─ small commit (feature/auth)  
└─ small commit (feature/api)  
└─ small commit (bugfix/cache)  
└─ small commit (feature/ui)  
└─ small commit (refactor/db)
```

Characteristics:

- └─ Commits directly to main
- └─ Short-lived branches (< 1 day)
- └─ Feature flags for incomplete features
- └─ Continuous testing & deployment
  - └─ Fast feedback loops

### 3. Code Review Process Architecture

#### Review Workflow

```
Developer creates feature branch
  ↓
Commits changes
  ↓
Pushes to repository
  ↓
Creates Pull Request
  ├── Fills template
  ├── References issue
  ├── Describes changes
  └── Requests reviewers
  ↓
Automated Checks Run
  ├── Linting
  ├── Unit tests
  ├── Build
  └── Security scan
  ↓
[PASS] → Continue | [FAIL] → Fix & recommit
  ↓
Reviewers Examine
  ├── Code quality
  ├── Design patterns
  ├── Test coverage
  ├── Documentation
  └── Performance
  ↓
[APPROVE] → Merge | [CHANGES REQUESTED] → Iterate
  ↓
Merge to main
  ├── Squash commits (optional)
  ├── Delete branch
  └── Trigger deployment
  ↓
Monitor in production
  └── Health checks & metrics
```

## Review Criteria Template

---

#### CODE QUALITY

- └ [ ] Code follows style guide
- └ [ ] No dead code
- └ [ ] Proper error handling
- └ [ ] Security best practices
- └ [ ] Performance acceptable

#### TESTING

- └ [ ] Unit tests added
- └ [ ] Integration tests pass
- └ [ ] Test coverage adequate
- └ [ ] Edge cases handled

#### DOCUMENTATION

- └ [ ] Code comments where needed
- └ [ ] Docstrings present
- └ [ ] README updated
- └ [ ] Migration guide (if needed)

#### DESIGN

- └ [ ] Architecture sound
- └ [ ] No breaking changes
- └ [ ] Backward compatibility
- └ [ ] Maintainability good

#### RISK ASSESSMENT

- └ [ ] Changes isolated
- └ [ ] Rollback plan exists
- └ [ ] Monitoring in place
- └ [ ] No data loss risk

## 4. Repository Management Patterns

### Branch Protection Strategies

#### Main Branch Rules:

- └─ Require pull request review
  - └─ Minimum 1 reviewer
- └─ Require status checks
  - └─ Build must pass
  - └─ Tests must pass
  - └─ Linter must pass
- └─ Require branches up to date
  - └─ Must be merged with main
- └─ Dismiss stale reviews
  - └─ If new commits pushed
- └─ Restrict who can push
  - └─ Only admins can force-push
- └─ Require CODEOWNERS approval
  - └─ For code ownership

## CODEOWNERS Configuration

```
# CODEOWNERS file structure

# Entire repo
* @platform-team

# API code
/src/api/ @api-team

# Database migrations
/migrations/ @database-team @platform-team

# Infrastructure
/terraform/ @infrastructure-team
/kubernetes/ @infrastructure-team

# Frontend
/frontend/ @frontend-team

# Specific files
Dockerfile @docker-team
*.tf @infrastructure-team
package.json @build-team

# New files default to
# whoever is listed first
```

## 5. Integration Patterns

### Cloud Build Integration

Repository Trigger:

```
└─ New commit to main
|
└─ Webhook to Cloud Build
|
└─ Read cloudbuild.yaml from repo
|
└─ Execute build steps
|   └─ Step 1: Build Docker image
|   └─ Step 2: Run tests
|   └─ Step 3: Push to registry
|   └─ Step 4: Deploy
|
└─ Report status to GitHub
|   └─ Build success/failure
|   └─ Test results
|   └─ Coverage report
|
└─ Comment on PR with results
    └─ "✅ All checks passed"
        or "❌ Build failed"
```

### Webhook Configuration

Webhook Event: Push to main

↓

HTTP POST to: <https://cloudbuild.googleapis.com/...>  
with payload:

```
{  
  "ref": "refs/heads/main",  
  "repository": "my-app",  
  "commit": "abc123...",  
  "author": "developer@company.com",  
  "message": "Add feature X"  
}
```

↓

Cloud Build receives

↓

Triggers build with repository content

↓

Executes `cloudbuild.yaml` steps

↓

Reports results back to repository

## 6. Repository Best Practices Architecture

### Commit Message Standard

Format: `<type>(<scope>): <subject>`

`<body>`

`<footer>`

Example:

`feat(auth): Add OAuth 2.0 support`

Implement OAuth 2.0 authentication flow  
to support third-party application access.

Adds:

- OAuth provider configuration
- Token generation and validation
- User linking with OAuth accounts

Fixes: #1234

Relates-to: #5678

## Commit Types

```
feat:    New feature
fix:     Bug fix
docs:    Documentation change
style:   Code style (no functional change)
refactor: Code refactoring
perf:    Performance improvement
test:    Test addition/modification
chore:   Build/tool/dependency change
ci:      CI/CD configuration change
```

---

## INTEGRATION PATTERNS

### 1. IAM + Terraform Integration

#### Service Account-Based Deployment

```
hcl
```

```

# Terraform configuration for CI/CD deployment

# Create service account for CI/CD
resource "google_service_account" "ci_cd" {
  account_id    = "ci-cd-deployer"
  display_name  = "CI/CD Deployment Service Account"
}

# Grant necessary roles
resource "google_project_iam_member" "ci_compute_admin" {
  project = var.project_id
  role    = "roles/compute.admin"
  member  = "serviceAccount:${google_service_account.ci_cd.email}"
}

resource "google_project_iam_member" "ci_storage_admin" {
  project = var.project_id
  role    = "roles/storage.admin"
  member  = "serviceAccount:${google_service_account.ci_cd.email}"
}

# Create key for authentication
resource "google_service_account_key" "ci_cd_key" {
  service_account_id = google_service_account.ci_cd.name
  public_key_type     = "TYPE_X509_PEM_FILE"
}

# Output for CI/CD system
output "ci_cd_key_json" {
  value     = base64decode(google_service_account_key.ci_cd_key.private_key)
  sensitive = true
}

```

## 2. Cloud Source Repositories + Cloud Build + Terraform

### Complete CI/CD Pipeline

## Repository Structure:

```
my-app/  
├─ src/                (application code)  
├─ terraform/          (infrastructure)  
│   ├─ main.tf  
│   ├─ variables.tf  
│   └─ outputs.tf  
├─ cloudbuild.yaml      (build pipeline)  
└─ .gitignore
```

cloudbuild.yaml:

steps:

# Step 1: Validate Terraform

```
- name: 'gcr.io/cloud-builders/terraform'  
  args:  
    - validate  
    - terraform/
```

# Step 2: Plan infrastructure changes

```
- name: 'gcr.io/cloud-builders/terraform'  
  args:  
    - plan  
    - -out=tfplan  
    - terraform/  
  env:  
    - 'GOOGLE_CREDENTIALS=/workspace/sa-key.json'
```

# Step 3: Build application

```
- name: 'gcr.io/cloud-builders/docker'  
  args:  
    - build  
    - -t  
    - gcr.io/$PROJECT_ID/my-app:$SHORT_SHA  
    - .
```

# Step 4: Push image

```
- name: 'gcr.io/cloud-builders/docker'  
  args:  
    - push  
    - gcr.io/$PROJECT_ID/my-app:$SHORT_SHA
```

# Step 5: Apply infrastructure

```
- name: 'gcr.io/cloud-builders/terraform'  
  args:
```

```
- apply
- tfplan
env:
  - 'GOOGLE_CREDENTIALS=/workspace/sa-key.json'

# Step 6: Update deployment
- name: 'gcr.io/cloud-builders/gke-deploy'
  args:
    - run
    - --filename=.
    - --image=gcr.io/${PROJECT_ID}/my-app:${SHORT_SHA}
    - --location=us-central1

images:
  - gcr.io/${PROJECT_ID}/my-app:${SHORT_SHA}
```

### 3. Marketplace + Terraform + Source Control

#### Infrastructure-as-Code for Marketplace Solutions

hcl

```
# Deploy marketplace solution with Terraform

resource "google_compute_instance_from_template" "wordpress" {
  name                = "wordpress-site"
  source_instance_template = data.google_compute_image.wordpress.self_link

  tags = ["http-server", "https-server"]

  # Add to version control
  labels = {
    deployed_by = "terraform"
    version     = "1.0"
    marketplace = "wordpress"
  }
}

# Data source from marketplace image
data "google_compute_image" "wordpress" {
  family = "wordpress-single"
  project = "click-to-deploy-images"
}

# Support infrastructure
resource "google_compute_firewall" "wordpress_http" {
  name      = "allow-wordpress-http"
  network   = "default"

  allow {
    protocol = "tcp"
    ports    = ["80", "443"]
  }

  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["http-server"]
}

# Backup configuration
resource "google_compute_disk" "wordpress_backup" {
  name                = "wordpress-backup"
  type                = "pd-standard"
  zone                = "us-central1-a"
  snapshot_schedule = google_compute_resource_policy.daily_backup.id
}
```

```
resource "google_compute_resource_policy" "daily_backup" {
  name    = "daily-backup"
  region = "us-central1"

  snapshot_schedule_policy {
    schedule {
      daily_schedule {
        days_in_cycle = 1
        start_time     = "03:00"
      }
    }
    retention_policy {
      max_retention_days = 30
    }
  }
}
```

## 4. Complete Implementation Flow

Developer Flow:

- └ Create feature branch in CSR
  - |
  - └ Make code changes
    - | └ Application code
    - | └ Terraform infrastructure
    - | └ Update tests
  - |
  - └ Commit with message
    - | └ Follows commit message standard
  - |
  - └ Push to repository
    - | └ Branch pushed to Cloud Source Repositories
  - |
  - └ Create Pull Request
    - | └ GitHub-like interface
    - | └ Reference issues
    - | └ Request reviewers
  - |
  - └ Automated Checks
    - | └ Cloud Build runs
    - | └ Terraform validate
    - | └ Application tests
    - | └ Security scan
    - | └ Report status
  - |
  - └ Code Review
    - | └ Reviewers examine
    - | └ Platform team reviews Terraform
    - | └ App team reviews code
    - | └ Security team reviews changes
  - |
  - └ Merge to main
    - | └ All checks pass
    - | └ Approval received
    - | └ Automatic squash & merge
  - |
  - └ Production Deployment
    - └ Cloud Build triggered
    - └ Terraform apply (infrastructure)
    - └ Docker build & push

- └ Kubernetes/Cloud Run deploy
- └ Health checks & monitoring

# ENTERPRISE IMPLEMENTATION

## 1. Multi-Environment Setup

### Environment Architecture

#### DEVELOPMENT

- └ Shared infrastructure
- └ Quick iteration
- └ Minimal compliance
- └ No PII/sensitive data

#### STAGING

- └ Production-like environment
- └ Full test coverage
- └ Performance testing
- └ Security testing

#### PRODUCTION

- └ High availability
- └ Full compliance
- └ Disaster recovery
- └ Audit logging enabled
- └ Limited access, strict controls

#### Repository Structure:

main (production code)

└ develop (staging integration)

└ feature branches (development)

## 2. Security & Compliance Framework

### Security Layers

#### Layer 1: Network

- └ VPC isolation
- └ Firewall rules
- └ Private services
- └ DDoS protection

#### Layer 2: Identity

- └ Service accounts
- └ IAM roles
- └ MFA enabled
- └ Access logging

#### Layer 3: Data

- └ Encryption at rest
- └ Encryption in transit
- └ Data classification
- └ Data retention policies

#### Layer 4: Application

- └ Input validation
- └ Output encoding
- └ Session management
- └ Security headers

#### Layer 5: Monitoring

- └ Cloud Audit Logs
- └ Security events
- └ Intrusion detection
- └ Vulnerability scanning

## Compliance Implementation

---

Compliance Requirement: SOC 2 Type II

Implementation:

- └ Service Account (technical control)
  - | └ Named clearly: "audit-reader-sa"
  - | └ Limited permissions
  - | └ Key rotation policy
- |
- └ IAM Policy (access control)
  - | └ Least privilege
  - | └ Separation of duties
  - | └ Regular review
- |
- └ Cloud Audit Logs (audit trail)
  - | └ Admin activity logged
  - | └ Data access logged
  - | └ Retention: 365 days
  - | └ Immutable storage
- |
- └ Monitoring (continuous verification)
  - | └ Alerts for unauthorized access
  - | └ Anomaly detection
  - | └ Regular reports

### 3. GitOps & Automation

#### GitOps Principles

Git Repository = Source of Truth

↓

- └ Pull from git
- |
- └ Compare with actual state
- |
- └ Sync if different
- |
- └ Repeat every N seconds

Benefits:

- └ All changes tracked in git
- └ Atomic rollback possible
- └ Audit trail preserved
- └ Disaster recovery via git
- └ Reproducible deployments

## Automated Deployment Flow

```
Developer commits to main
  ↓
Git webhook triggered
  ↓
Cloud Build receives event
  ↓
Pipeline starts:
  ├── Checkout code
  ├── Run tests
  ├── Build artifacts
  ├── Terraform validate
  ├── Terraform plan (staging)
  ├── Manual approval (production)
  ├── Terraform apply
  ├── Deploy application
  └── Run smoke tests
  ↓
Deployment complete
  ├── Metrics monitored
  ├── Logs aggregated
  └── Alerts configured
```

## 4. Disaster Recovery Strategy

### RTO & RPO Planning

#### RT0 (Recovery Time Objective)

- └ How fast to restore services
- └ Development: 8 hours
- └ Staging: 4 hours
- └ Production: 1 hour

#### RPO (Recovery Point Objective)

- └ How much data loss acceptable
- └ Development: 24 hours
- └ Staging: 12 hours
- └ Production: 1 hour

#### Achieving Production SLA:

- └ Infrastructure as Code (redploy in minutes)
- └ Automated backups (hourly)
- └ Replication (multiple regions)
- └ Automated failover (DNS switch)
- └ Testing (quarterly DR drills)

## Recovery Procedures

---

## Scenario: Data Loss

### Detection

- └─ Monitoring alert
- └─ User report
- └─ Manual check

### Response (0-15 minutes)

- └─ Identify loss scope
- └─ Notify stakeholders
- └─ Prepare rollback

### Recovery (15-45 minutes)

- └─ Restore from backup
- └─ Validate data integrity
- └─ Switch to restored version

### Verification (45-60 minutes)

- └─ Smoke tests pass
- └─ Production traffic restored
- └─ Post-mortem begins

### Post-Incident

- └─ Root cause analysis
- └─ Preventive measures
- └─ Process improvements
- └─ Team training

---

## ADVANCED TOPICS

### 1. Cost Optimization Strategies

#### Cost Analysis Framework

---

compute.googleapis.com (40%)

- └─ Compute Engine instances
- └─ GKE cluster nodes
- └─ Cloud Functions execution

storage.googleapis.com (30%)

- └─ Persistent disks
- └─ Cloud Storage
- └─ Database storage

network.googleapis.com (20%)

- └─ Data egress
- └─ Load balancer
- └─ Cloud Interconnect

other (10%)

- └─ Cloud Build
- └─ Cloud Logging
- └─ Cloud Monitoring

## Optimization Techniques

---

### 1. Right-Sizing

Over-provisioned → e2-medium  
→ e2-small

Saves: 40-50%

### 2. Committed Use Discounts

Pay upfront for 1-3 year commitment

Saves: 25-70%

### 3. Preemptible Resources

Short-lived, interruptible instances

Saves: 70-90%

Use for: Batch jobs, testing

### 4. Auto-Scaling

Scale based on demand

Avoid: Idle resources

### 5. Scheduled Scaling

Scale down overnight

Scale up for business hours

Saves: 30-50%

## 2. Performance Optimization

### Terraform Performance

#### Optimization 1: Parallel Operations

Default: Terraform makes 10 concurrent API calls

– parallelism = 10

Increase for large deployments:

terraform apply -parallelism=20

(Caution: API rate limits)

#### Optimization 2: Partial Applies

Deploy subset of infrastructure:

terraform apply -target=module.network

(Use when specific component needs update)

#### Optimization 3: Targeted Refresh

Refresh specific resources only:

terraform refresh -target=google\_compute\_instance.app

(Faster than full refresh)

## Repository Performance

Large Repository Optimization:

- └─ Shallow clones
  - | `git clone --depth 1` (faster, less history)
  - |
- └─ Sparse checkout
  - | `git sparse-checkout` (don't download everything)
  - |
- └─ LFS for large files
  - | `git lfs` (separate storage for binaries)
  - |
- └─ Repository pruning
  - | Cleanup old branches and tags regularly

## 3. Testing & Validation

### Infrastructure Testing

Unit Tests (Terraform)

- └─ `terraform validate` (syntax)
- └─ `terraform plan` (logic)
- └─ Linters (style)

Integration Tests

- └─ Deploy to test environment
- └─ Verify resources created correctly
- └─ Test inter-resource communication

Compliance Tests

- └─ Verify security settings
- └─ Check firewall rules
- └─ Validate encryption

Policy Tests

- └─ Ensure naming conventions
- └─ Check tagging
- └─ Verify quotas

### Security Testing

---

### Pre-commit Checks

- └ Secret scanning (credentials in code)
- └ Dependency scanning (vulnerable libraries)
- └ Code quality (security issues)

### Build-Time Checks

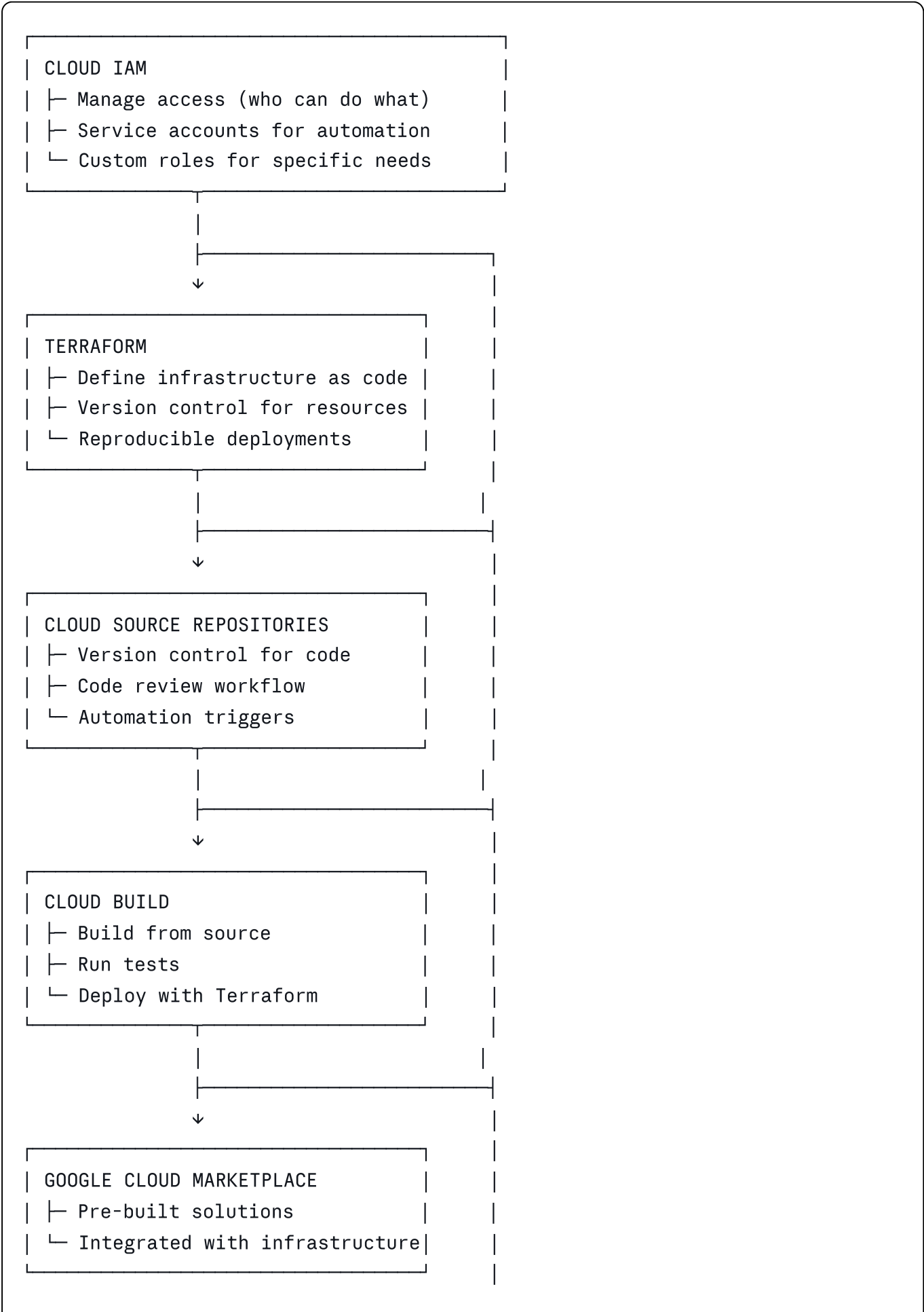
- └ Container scanning
- └ SBOM generation
- └ License compliance

### Runtime Checks

- └ Configuration auditing
  - └ Runtime security
  - └ Anomaly detection
-

# CONCLUSION

## Integration Summary



|| ALL SECURED WITH IAM  
|| SERVICE ACCOUNTS + LEAST PRIVILEGE

## Key Takeaways

1. **IAM is Foundation:** Every resource needs appropriate access control
2. **Terraform Enables Scale:** Define infrastructure for reproducibility
3. **Marketplace Accelerates:** Use pre-built solutions for faster delivery
4. **Source Control Tracks:** All changes tracked for compliance & rollback
5. **Integration Multiplies Value:** These tools work together for enterprise automation

---

## End of Deep Dive Tutorial

*This tutorial covers advanced concepts and patterns without step-by-step labs. Refer to official Google Cloud documentation for implementation details.*